Guia de Documentação do Trabalho

Codificação de Linha Manchester em VHDL (Vivado)

Comunicação de Dados - Sistemas Reconfiguráveis

Apresentação

Este guia orienta o conteúdo esperado em cada documento da entrega do trabalho. A entrega é composta por quatro tipos de documento, cada um com um propósito específico:

Documento	Pergunta que responde	Local
READMEs	"O que é isso e por onde começo?"	Raiz e cada projeto
Tutoriais	"Como eu reproduzo isso?"	`docs/` de cada projeto
Documentação	"Como isso funciona internamente?"	`docs/` de cada projeto
Relatório Técnico	"Por que foi feito assim e o que foi descoberto?"	`relatorio_final/`

1 README

O README é o ponto de entrada do repositório. É o primeiro arquivo lido por quem chega ao projeto, e deve dar uma visão geral rápida do que está ali e como navegar.

1.1 README da raiz do repositório

Identifica o trabalho como um todo e **descreve a organização do repositório**, funcionando como mapa para quem chega ao projeto. Deve conter, no mínimo:

- Título do trabalho;
- Disciplina, semestre e nome do(a) professor(a);
- Integrantes do grupo;
- Breve descrição do trabalho (1 a 2 parágrafos);
- **Estrutura do repositório:** árvore de pastas explicando o que tem em cada lugar;
- Índice apontando para cada projeto e para o relatório final;
- Lista de ferramentas utilizadas (Vivado, versão, placa FPGA, etc.);
- Como começar (link para o primeiro tutorial a ser seguido).

Exemplo de estrutura do README raiz

```
# Codificação Manchester em VHDL

Trabalho da disciplina de Comunicação de Dados - Sistemas
Reconfiguraveis, ministrada por <nome do professor>.
Semestre <ano/semestre>.

## Integrantes
```

````
- Nome 1
- Nome 2

## Descrição
Este trabalho implementa, em VHDL, dois codificadores de linha:
Manchester (IEEE 802.3) e Manchester Diferencial. O codigo foi
simulado no Vivado Simulator e implementado na placa Basys 3.

## Estrutura do repositório

```
modulacao_manchester/
|-- manchester/              Projeto Manchester (auto-contido)
|-- manchester_diferencial/  Projeto Manchester Diferencial
`-- relatorio_final/    Relatório técnico consolidado e vídeo demo
```

Cada projeto possui sua própria pasta `docs/` com tutoriais e
documentação técnica, e uma pasta `saidas/` com as saídas
PMOD (osciloscópio) e VGA.

## Projetos
- [Manchester](./manchester/) - bit dado pelo sentido da
  transição no meio do bit
- [Manchester Diferencial](./manchester_diferencial/) - bit
  dado pela presença de transição no início do bit

## Relatório
- [Relatório técnico final](./relatorio_final/relatorio.pdf)
- [Vídeo de demonstração](./relatorio_final/video_demo.mp4)

## Ferramentas utilizadas
- Vivado 2023.1
- Placa Basys 3 (Artix-7)
- Osciloscópio (para a saída PMOD)

## Como começar
Recomendamos iniciar pelo projeto Manchester. Acesse a pasta
[manchester/](./manchester/) e siga o README local.
````

1.2 README de cada projeto (manchester e manchester_diferencial)

Cada projeto deve ter o próprio README, focado naquele projeto específico e **descrevendo a estrutura local da pasta**. Quem entrar diretamente na pasta de um projeto (sem ter lido o README raiz) deve conseguir se localizar. Deve conter:

- Nome e descrição curta do projeto (qual codificação implementa);
- Breve explicação teórica da codificação (2 a 3 linhas), indicando a convenção adotada;
- **Estrutura da pasta:** árvore mostrando o que tem ali dentro;
- Lista dos arquivos de código com explicação breve do que cada um faz;

- Atalhos (links) para os tutoriais e a documentação dentro de `docs/`;
- **Por onde começar:** trilha de leitura indicando a ordem dos documentos a serem seguidos para reproduzir o projeto;
- Links para as saídas PMOD e VGA do projeto;
- **Próximo passo:** link para o próximo projeto do trabalho (apenas no README do `manchester/`; o `manchester_diferencial/` aponta para o relatório final).

Exemplo de README de um projeto

````
# Codificação Manchester

Implementação em VHDL do código Manchester (convenção IEEE
802.3), em que o bit é representado pelo sentido da transição
no meio do intervalo de bit (0 = descida, 1 = subida).

## Estrutura desta pasta

```
manchester/
|-- src/               Código-fonte VHDL do circuito
|-- sim/               Testbench para simulação
|-- constraints/       Mapeamento dos pinos da FPGA (.xdc)
|-- vivado_project/    Projeto Vivado pronto para abrir
|-- docs/              Tutoriais e documentação técnica
`-- saidas/            Saídas PMOD e VGA
```

## Arquivos de código
- `src/manchester.vhd` - código-fonte do circuito principal
- `sim/manchester_tb.vhd` - testbench para simulação
- `constraints/manchester.xdc` - mapeamento dos pinos da FPGA

## Documentação
- [Tutorial de simulação](./docs/tutorial_simulacao.pdf) -
como simular no Vivado passo a passo
- [Tutorial de gravação na placa](./docs/tutorial_placa.pdf) -
como sintetizar, gerar bitstream e gravar na FPGA
- [Documentação técnica](./docs/documentacao_projeto.pdf) -
diagrama de blocos, descrição das portas e funcionamento

## Por onde começar

Para uma **simulação rápida** (apenas ver funcionando):

1. Abrir o Vivado e carregar `vivado_project/manchester.xpr`
2. Clicar em `Run Simulation` -> `Run Behavioral Simulation`

Para **reproduzir o projeto do zero**, siga esta ordem:

1. **Entenda o circuito** lendo a
[documentação técnica](./docs/documentacao_projeto.pdf)
2. **Simule no Vivado** seguindo o
[tutorial de simulação](./docs/tutorial_simulacao.pdf)
````

```
3. **Grave na placa** seguindo o
[tutorial de gravação](./docs/tutorial_placa.pdf)
4. **Implemente as atividades**
[saídas PMOD e VGA](./saidas/)

## Saídas PMOD e VGA
- [Saída pelo PMOD para osciloscópio](./saidas/pmod_osciloscopio/)
- [Saída VGA com cores pulsantes](./saidas/vga/)

## Próximo passo

Após concluir este projeto, siga para a
[Codificação Manchester Diferencial](../manchester_diferencial/),
que implementa a outra versão estudada no trabalho.
```

Exemplo de README do manchester_diferencial (segundo projeto)

````
# Codificação Manchester Diferencial

Implementação em VHDL do código Manchester Diferencial, em que
sempre há transição no meio do bit e o valor do bit é dado
pela transição no início do intervalo (0 = inversão do nível,
1 = mantém o nível anterior). Nível após o reset: baixo.

## Estrutura desta pasta

```
manchester_diferencial/
|-- src/               Código-fonte VHDL do circuito
|-- sim/               Testbench para simulação
|-- constraints/       Mapeamento dos pinos da FPGA (.xdc)
|-- vivado_project/    Projeto Vivado pronto para abrir
|-- docs/              Tutoriais e documentação técnica
`-- saidas/            Saídas PMOD e VGA
```

## Arquivos de código
- `src/manchester_dif.vhd` - código-fonte do circuito principal
- `sim/manchester_dif_tb.vhd` - testbench para simulação
- `constraints/manchester_dif.xdc` - mapeamento dos pinos

## Documentação
- [Tutorial de simulação](./docs/tutorial_simulacao.pdf) -
como simular no Vivado passo a passo
- [Tutorial de gravação na placa](./docs/tutorial_placa.pdf) -
como sintetizar, gerar bitstream e gravar na FPGA
- [Documentação técnica](./docs/documentacao_projeto.pdf) -
diagrama de blocos, descrição das portas e funcionamento

## Por onde começar

Para uma **simulação rápida** (apenas ver funcionando):

1. Abrir o Vivado e carregar `vivado_project/manchester_dif.xpr`
2. Clicar em `Run Simulation` -> `Run Behavioral Simulation`

Para **reproduzir o projeto do zero**, siga esta ordem:

1. **Entenda o circuito** lendo a
[documentação técnica](./docs/documentacao_projeto.pdf)
2. **Compare com o Manchester** (visto no projeto anterior)
para identificar as diferenças na lógica de transição
3. **Simule no Vivado** seguindo o
[tutorial de simulação](./docs/tutorial_simulacao.pdf)
4. **Grave na placa** seguindo o
[tutorial de gravação](./docs/tutorial_placa.pdf)
5. **Implemente as atividades**
[saídas PMOD e VGA](./saidas/)

## Saídas PMOD e VGA
````

```
- [Saída pelo PMOD para osciloscópio](./saidas/pmod_osciloscopio/)
- [Saída VGA com cores pulsantes](./saidas/vga/)

## Próximo passo

Você concluiu os dois projetos do trabalho. Agora acesse o
[Relatório técnico final](../relatorio_final/relatorio.pdf)
para ver a análise comparativa entre os dois códigos, os
resultados obtidos e as conclusões do grupo.
```

1.3 README das pastas `saidas/`

Cada saída dentro de `saidas/` (PMOD e VGA) também deve ter um pequeno README explicando o que foi feito. Deve conter:

- O que a saída implementa e por que;
- Estrutura local da pasta;
- Como reproduzir (conexões físicas necessárias, equipamentos);
- Link para o tutorial específico em `docs/`.

Exemplo de README de saída (PMOD/osciloscópio)

```
# Saída pelo PMOD (Osciloscópio)

Disponibiliza o sinal codificado e o dado original em pinos do
conector PMOD da placa, permitindo visualizar as duas formas de
onda simultaneamente em um osciloscópio físico.

## Estrutura desta pasta

‘‘‘
pmod_osciloscopio/
|-- src/             Código VHDL adaptado com saída no PMOD
|-- constraints/     Constraint com mapeamento do pino PMOD
‘-- docs/            Tutorial e capturas do osciloscópio
‘‘‘

## Equipamentos necessários
- Placa FPGA Basys 3
- Osciloscópio de dois canais
- Cabo BNC ou pontas de prova para conectar ao PMOD

## Como reproduzir
Ver o [tutorial completo](./docs/tutorial_pmod.pdf), que descreve:
1. Conexão física do osciloscópio ao pino do PMOD
2. Gravação do bitstream
3. Configuração da escala do osciloscópio
4. Captura da forma de onda
```

2 Tutoriais (`docs/tutorial_*.pdf`)

Os tutoriais são documentos **operacionais**: explicam **como fazer**, passo a passo. Não precisam justificar decisões nem explicar conceitos, apenas guiar a execução. Devem ser autoexplicativos a ponto de outra pessoa conseguir reproduzir o trabalho sem precisar perguntar nada.

2.1 tutorial_simulacao.pdf

Passo a passo da simulação no Vivado. Deve conter:

Roteiro mínimo do tutorial de simulação

1. **Pré-requisitos:** versão do Vivado, sistema operacional;
2. **Abertura do projeto:** como abrir o projeto Vivado da pasta `vivado_project/`;
3. **Estrutura dos arquivos no Vivado:** identificar onde estão o *source*, o *testbench* e o *top module*;
4. **Configuração da simulação:** definir o testbench como *top*, ajustar tempo de simulação;
5. **Execução da simulação:** `Run Behavioral Simulation`;
6. **Visualização do waveform:** como adicionar sinais, ajustar a escala de tempo, ler a forma de onda;
7. **Resultado esperado:** indicar o que deve aparecer na simulação (com print de referência).

2.2 tutorial_placa.pdf

Passo a passo da gravação na placa FPGA. Deve conter:

Roteiro mínimo do tutorial de placa

1. **Pré-requisitos:** placa, cabo, drivers, Vivado;
2. **Síntese:** executar `Run Synthesis`;
3. **Implementação:** executar `Run Implementation`;
4. **Geração do bitstream:** `Generate Bitstream`;
5. **Conexão da placa:** como conectar via USB, abrir o `Hardware Manager`;
6. **Programação:** `Program Device`;
7. **Mapeamento de uso na placa:** quais switches controlam o quê, como o LED ativo indica a posição e o nível do sinal codificado, qual botão reseta;
8. **Como testar:** sugestão de sequências de bits para configurar nos switches (ex: todos 0, todos 1, alternados) e o que esperar nos LEDs em cada metade do bit (com fotos ou vídeo de referência).

Observação importante: tutoriais devem ter **prints e/ou fotos** ilustrando cada etapa. Texto puro não funciona bem como tutorial.

3 Documentação do Projeto (`docs/documentacao_projeto.pdf`)

A documentação descreve **como o projeto funciona internamente**. É o documento que alguém leria para entender, modificar ou estender o circuito. Diferente do tutorial, aqui o foco está em **conceito** e não em execução.

Conteúdo esperado

1. **Visão geral do circuito:** o que ele faz e qual o código de linha que implementa (com a convenção adotada);
2. **Diagrama de blocos:** representação visual dos componentes internos e do fluxo de dados (clock, contador de ciclos, detecção do meio do bit, seleção do bit, registrador do nível anterior, saída);
3. **Interface (entradas e saídas):** tabela com todas as portas do módulo, seus tipos e funções. Exemplo:

Porta	Tipo	Função
clk	std_logic	Sinal de clock da FPGA
rst	std_logic	Reset (botão BTNC)
data_in	std_logic_vector(15 downto 0)	Sequência de bits (switches)
data_out	std_logic	Sinal codificado (PMOD)
led	std_logic_vector(15 downto 0)	LEDs (bit em transmissão)

4. **Parâmetros (generics):** valores configuráveis e o que controlam (ex: `CYCLES_PER_BIT`, que deve ser par);
5. **Funcionamento interno:** explicação do que acontece dentro do código (lógica do contador, detecção do meio do bit, lógica de seleção do bit, regra de transição no início do bit do Manchester Diferencial, nível após o *reset*, etc.);
6. **Mapeamento físico na placa:** quais pinos da FPGA estão conectados a quais sinais (switches, LEDs, botões);
7. **Dependências e ambiente:** versão do Vivado, modelo da placa, bibliotecas VHDL utilizadas.

Tom de escrita: a documentação é escrita no **presente** e de forma **impessoal**, como uma referência técnica.

Exemplo de trecho da documentação

"O módulo `manchester` recebe um vetor de 16 bits pela porta `data_in` e gera, na saída `data_out`, o sinal codificado em Manchester segundo a convenção IEEE 802.3. A duração de cada bit é definida pelo generic `CYCLES_PER_BIT`, que controla um contador interno responsável por determinar o instante do meio do bit, no qual a saída é invertida, e o instante de troca de bit."

4 Relatório Técnico (`relatorio_final/relatorio.pdf`)

O relatório é o **documento avaliativo** do trabalho. Diferente da documentação (que descreve o sistema), o relatório **narra o processo** do grupo: o que foi feito, como foi feito, quais resultados foram obtidos e quais conclusões foram tiradas.

Estrutura sugerida

1. **Capa:** título do trabalho, integrantes, disciplina, professor, instituição, semestre;
2. **Resumo:** breve parágrafo descrevendo o que foi feito e os principais resultados (5 a 10 linhas);
3. **Introdução:** contexto da codificação de linha, motivação do trabalho, objetivos gerais e específicos;
4. **Fundamentação teórica:** conceitos teóricos dos códigos Manchester e Manchester Diferencial, com diagramas: convenções existentes, autossincronização, componente DC e largura de banda. Citar referências bibliográficas;
5. **Metodologia:** como o trabalho foi organizado (etapas: estudo → implementação em VHDL → simulação → síntese → gravação na placa). Ferramentas utilizadas;
6. **Desenvolvimento:** descrição do que foi implementado em cada projeto. Decisões de projeto (por que escolheram tal abordagem, qual a duração do bit adotada, quais dificuldades encontraram e como resolveram);
7. **Resultados:**
   - Prints das simulações com identificação dos sinais;
   - Fotos da placa em funcionamento;
   - Capturas do osciloscópio (saída PMOD);
   - Fotos ou vídeo da tela VGA (saída VGA).
8. **Análise e discussão:**
   - Comparação entre Manchester e Manchester Diferencial (vantagens, desvantagens, características do sinal, comportamento com inversão de polaridade);
   - Comparação dos códigos Manchester com os códigos NRZ (número de transições por bit, sincronismo, largura de banda necessária);
   - Comparação entre o resultado da simulação e o resultado obtido na placa;
   - Interpretação dos resultados observados.
9. **Conclusões:** principais aprendizados, limitações observadas, possíveis melhorias ou trabalhos futuros;
10. **Referências bibliográficas:** livros, artigos, datasheets e materiais consultados (formato ABNT ou IEEE).

Tom de escrita: o relatório é escrito no **passado** e de forma **impessoal**, evitando uso da primeira pessoa.

Exemplo de trecho do relatório

"Para implementar o Manchester Diferencial, optou-se por utilizar um flip-flop interno que armazena o nível do sinal de saída. No início de cada bit, a lógica verifica o valor do bit atual: caso seja 0, o nível é invertido; caso seja 1, o nível é mantido. Na metade do intervalo, o nível é sempre invertido. Essa abordagem se mostrou eficiente porque (...)"

5 Comparativo entre os documentos

A tabela abaixo resume as principais diferenças entre os quatro tipos de documento, para evitar confusão sobre o que colocar em cada lugar:

	README	**Documentação**	**Relatório**
Propósito	Orientação e navegação	Referência técnica do sistema	Avaliação do trabalho realizado
Público	Quem chega ao repositório	Quem vai usar ou modificar	Professor (avaliador)
Tempo verbal	Presente	Presente	Passado
Pessoa	Impessoal	Impessoal	Impessoal
Tamanho	Curto	Médio	Longo
Atualiza com o código?	Sim	Sim	Não (registro do semestre)
Inclui resultados?	Não	Não (só descreve o que faz)	Sim (prints, fotos, análises)

Resumo rápido

- **README** = "Estou no projeto. O que tem aqui e por onde começo?"
- **Tutorial** = "Como eu reproduzo isso passo a passo?"
- **Documentação** = "Como esse circuito funciona internamente?"
- **Relatório** = "O que foi feito, como foi feito e o que foi descoberto?"

Dicas finais

- Comece pelos READMEs (são curtos e organizam o resto);
- Escreva os tutoriais **enquanto** executam as etapas (não depois), assim nada é esquecido;
- A documentação pode ser construída em paralelo ao desenvolvimento do código;
- Deixe o relatório por último, mas vá guardando prints, fotos e observações desde o início;
- Releia tudo no final pensando: *"alguém de outro grupo entenderia isso sem perguntar nada?"*